

20 分钟讲稿 · 把判断说出来，让 skill 永远记住

配套 index.html 使用。节奏建议：开场 1 分 → Part 1 五层 5 分 → Part 2 引擎 5 分 → Part 3 飞轮 2 分 → Case SpaceX 6 分 → 收束 1 分。每段末尾的 **〔切换：...〕** 是页面操作提示。

0 · 开场（1 分钟，Hero 区）

〔页面：停在首页 Hero〕

各位好，今天这 20 分钟，我想跟大家讲一件很小、但我相信会改变我们组织形态的事——**怎么把一位资深同事嘴里那句“该这样改”，永久地变成系统的一部分。**

主题就一句话：**把判断说出来，让 skill 永远记住。**

我们今天要做的，**不是讲 AI 多牛**，而是回答一个很具体的问题——为什么我们公司里那些最值钱的判断，每天都在被一次性消耗掉？以及，工程上要做什么，才能让它停止被消耗。

我会分四块讲：

1. 我们参考的理论底座——Bttr 提的五层循环；
2. 我们做的 MVP——指令驱动 Skill 进化引擎，四个动作；
3. 把这两件事咬合起来，是一个组织级飞轮；
4. 用 SpaceX 招股书调研当一个完整案例，把整条循环跑给大家看。

〔切换：点击导航 CH 01 五层循环〕

1 · Bttr 五层循环（5 分钟）

1.1 立论：公司不是一张组织架构图（45 秒）

Bttr 那篇 *Burn Tokens, Not Headcount* 的开篇就把传统组织拆穿了——今天公司像罗马军团那样的层级结构，不是因为这种形态最优，而是低带宽沟通时代的应对机制（coping mechanism）。AI 把信息流

通成本压到接近零，"必须有中层来协调"的前提就消失了。

它给的替代叙述就一句：

"A company is not an org chart. It is a set of recursive, self-improving loops."

公司不是一张组织架构图，它是一组递归的、自我改进的循环。

注意三个词：循环（loop）、递归（recursive）、自我改进（self-improving）。这意味着我们不再问"我们要几个部门"，而是问"我们有几个核心循环、循环每跑一轮是不是变得更聪明"。

1.2 五层 + 一个中心（3 分 30 秒）

〔页面：滚到 1.0 末尾的 6 个 mini 卡片，再依次滚到 1.1–1.6〕

Bttr 给一个"能自我改进的循环"下了最小工程定义，五层 + 一个中心：

① **Sensor 感知层**：循环的输入感官——客户邮件、工单、commit、监报告警。一句话很硬：**循环只能作用于它能感知到的事**。一件事如果没被传感器接住，对系统就是没发生过。

② **Policy 策略层**：定义"什么动作 agent 可以自己拍板，什么必须升级"。它是工程化的硬约束，不是 prompt 里那种软约束。Bttr 原话——*the line between autonomy and oversight is set here, not at the prompt*。

③ **Tool 工具层**：所有"动手做事"的能力被暴露成确定性 API。Bttr 的形容词是 "boring"，这是高度褒义——**工具层越无聊越好**。决策面靠模型推理，工具面靠死板可靠。

④ **Quality Gate 质量闸门**：自动安全过滤 + 高风险动作的人工审核。它让"自治"变得可负担——没有闸门，要么不敢放，要么放开就出事。

⑤ **Learning Mechanism 学习机制**：这是 Bttr 说的"多数公司缺的那一层"。系统看着自己在哪里失败、写补丁、上线。"By morning it knows more than last night."

中心 · Company Context 公司语境：让公司之所以是这家公司的隐性知识，要被显式写下来，让循环里每个 agent 都能读到。这一句话最重要：

Legibility is the unlock. The company is a brain in the middle. Humans sit at the edge.

可读性是一切的解锁开关。公司是中间一颗大脑，人坐在边缘。

注意这是反转的——传统观是人在中间、公司架构服务于人的协作；AI Native 是反过来的，**大脑在中间**，人坐在边缘做新情境下的判断、谈判、品味。

1.3 钉子句 (45 秒)

〔页面：滚到 1.7 大图 + 钉子句〕

Bttr 文章的钉子句是这一句：

"If you were building your company today, would you start it in this shape? If you are small enough, you have no excuse."

它没让你立刻把公司翻新一遍，但它逼你回答——你今天做的每一件重复的事，能不能用"五层 + 一个中心"重描一遍。

我们的回答是：可以。下面就是我们挑了一条循环——投研调研循环——在它身上落地的最小工程。

〔切换：点击导航 CH 02 进化引擎〕

2 · 指令驱动 Skill 进化引擎 (5 分钟)

2.1 一句话方案 (1 分钟)

〔页面：2.0 一句话方案〕

MVP 的立场就一段话：

分析师不再埋头改 agent 的产出，**而是用一句自然语言指令让 agent 去改**——"承付义务这段太浅，拆出期限分布，重点说近两年占比"。这句指令本身就是最高质量的训练信号：**它无损地说清了"哪里不对、该怎么改、为什么"**。系统把这些指令攒起来，让反思器周期性地提炼出反复出现的模式，改写进 skill。下次同类调研，agent 第一次产出就已经包含了这个判断。

为什么不要"直接改产出"那种？因为：

"直接改产出只留下 diff，"为什么"丢了，得靠 agent 反推——有损、可能推错。**指令驱动里**，"为什么"是人亲口说的——无损、零推断，还省掉整个归因环节。"

底层纪律就一条：**改东西必须说出来，不能埋头自己改。**

2.2 四个动作 (4 分钟)

〔页面：依次滚到 2.1 / 2.2 / 2.3 / 2.4〕

动作 ①·记录指令 (Sensor) ——agent 每收到一条修改指令，自己调一个 `log_instruction()` 写库。判断标准就一条："这条消息是在让我修改刚才的产出吗？"是 → 记，否 → 跳过。**宁可多记，不可漏记**。这就是在补 Sensor 层的传感器。

动作 ②·入库 (Context) ——写进 SQLite 的 `instructions` 表。整张表只有 9 个字段，最关键的是 `instruction` 这一栏，**人说的原话，一字不改**。MVP 砍掉了所有需要"推断"的字段：`why_reason`、`severity`、`embedding` 全砍掉。一句话——**没有任何字段需要 AI 二次推断**，因为指令本身就是 `why`。这是这套设计最干净的地方。

动作 ③·反思器提炼 (Learning) ——按周或攒够 N 条触发，把同类指令喂给反思器。Prompt 的核心是两条规则：

- 找出**反复出现 ≥ 2 次**的模式；
- **忽略一次性的特例**。

为什么强调这两条？因为一次性的指令可能只是当下情境的特殊需求，硬塞进 skill 反而把所有未来任务绑架了。**只有反复出现的模式，才说明这是一个普遍判断**。

动作 ④·验证后进化 (Quality Gate) ——补丁不直接上线，要过两道闸：

1. **留出集**：拿一组黄金终稿做基线，新版必须 $\geq$ 基线分；
2. **人工一键确认**：skill 变更是高风险动作，人在闭环。

这里有一句业界铁律：**自我改进只在结果可验证处有效**。让一个系统改自己是危险的——除非你有一把客观的尺子量它改完之后是不是真的变好了。留出集就是那把尺子。

2.3 跟上一版的对比(30 秒)

〔页面：2.5 全景图 + 末尾绿色 callout〕

跟更早的设想比，这版砍掉了归因 **agent**、**diff 捕获**、**混合确认**、**severity 推断**——一句话：**指令是一手信号，直接用**。

〔切换：点击导航 CH 03 组织飞轮〕

3 · 组织飞轮（2 分钟）

3.1 五层映射（30 秒）

〔页面：3.1 映射表〕

把前两章合起来，会发现一件让人愉悦的事——进化引擎不是五层之外的另一个东西，它就是五层在投研业务上的实例化：

- 记录指令 → Sensor；
- "什么算指令"的规则 → Policy；
- log_instruction / SQLite / 反思器 prompt → Tool；
- 留出集 + 人工确认 → Quality Gate；
- 反思器 + 进化 → Learning；
- instructions 表 + skill 内容本身 → Company Context。

做一个能让 skill 进化的系统、做一个 AI Native 组织，工程上是同一件事的不同切片。

3.2 飞轮（1 分 30 秒）

〔页面：3.2 飞轮 SVG〕

每跑完一圈，下一圈起点都更高，因为：

1. skill 比上一版多了一条规则——v4 比 v3 知道得更多；
2. 分析师的注意力被释放——上一轮被改的那段，下一轮不用再改了。

这是 Bttr 那句 **slope vs level** 的具体形态——水平上每一次循环都在做事，斜率上每一次循环都在变聪明。公司的真正复利不来自任何一次循环本身，而来自循环的斜率。

〔切换：点击导航 CASE SpaceX 完整案例〕

4 · SpaceX 案例 · 完整循环走读（6 分钟）

数据来源声明：数据来自 ai_brain/PROJECTS/spacex_s1/，提交日 2026-05-20，财务截至 2026-03-31。下面要看到的"agent 行为""指令""diff"用于呈现完整 workflow 走查，帮助大家理解协作链路。

4.1 任务发起 (30 秒)

〔页面：STAGE 0〕

任务：基于 SpaceX 提交的 Form S-1，完成投资判断研究报告。当前用的是 prospectus_skill v3，里面有 6 条"拷问清单"规则——分部、利润率、客户集中度、关联交易、风险因子、现金流 vs 净利润。注意：v3 此时没有任何关于"承付义务怎么写"的规则。

4.2 第一轮：v3 的产出 (1 分钟)

〔页面：STAGE 1 v3 产出气泡〕

agent 用 v3 写出来的"承付义务"段，所有信息都对、所有数字都真——196 亿 EchoStar 收购、291 亿未偿债务、58 亿租赁——但是它没用。它读起来像"我把 S-1 翻译了一遍"，不像一份"我帮你想清楚 SpaceX 现金流压力有多大"的判断。

具体不够在哪里？没有期限分布、没有结构拆分、没有近两年占比、没有"那这又怎样"。

4.3 一句修改指令 (45 秒)

〔页面：STAGE 2 instruction-box〕

按 MVP 纪律——改东西必须说出来——分析师没有动手改，只在对话框里说了一句：

"承付义务这段太浅，拆出期限分布，重点说近两年占比和它对现金流的含义。"

agent 做两件事：① 自动调 log_instruction() 写进 instructions 表；② 在本次对话里按指令重写。分析师没有亲自改任何文字，她只是把判断说出来。

4.4 数据库里这条记录长什么样 (30 秒)

〔页面：STAGE 3 JSON〕

注意几个要点：instruction 字段一字不改，是一手 why，不需要二次推断；agent_output_before 存了原产出给反思器作上下文；promoted_to_skill = false 防重复消费。

4.5 一周后·反思器归纳 (1 分 30 秒)

〔页面：STAGE 4 6 条指令清单 + JSON 输出〕

到周日晚上，反思器读到了这一周里 6 条性质相似的指令——研发费、CapEx、AI 收入分部、承付义务（两条）、长期债务，6 条落在 6 个不同章节，但底层模式一致：分析师反复要求"把单一总额拆成构成

+ 期限"。

反思器没有为每条单独加规则，它做的是**抽象**——从 6 条指令里蒸馏出一条最一般的规则：

"大额数字必拆结构。任何单一总额（承付义务、研发费、CapEx、收入、长期债务）出现时，必须追问其构成与期限分布：近 1-2 年占比、按分部/用途/币种拆分。总额掩盖节奏，结构揭示压力。"

这就是反思器的价值——不是把指令复读进 skill，而是从指令里抽出可复用的判断模板。

4.6 验证闸 · 留出集打分（30 秒）

〔页面：STAGE 5 score-table〕

候选补丁过留出集——3 篇黄金终稿（SpaceX、OpenAI、Stripe），LLM-as-judge 三个维度打分。结果：**v3 平均 0.78，v4 候选 0.85**。≥ 基线，进入待上线。分析师周一上午看了一眼 diff，一键确认。

4.7 skill v3 → v4 上线（30 秒）

〔页面：STAGE 6 diff〕

skill 文件的真实 diff——拷问清单从 6 条变 7 条，新增第 7 条"大额数字必拆结构"，并且**追溯到 6 条 instruction id**—— #12, #15, #19, #23, #24, #27。整条因果链可审计、可回滚。

4.8 第二轮 · v4 第一次产出（1 分 15 秒）

〔页面：STAGE 7，先 v4 标签页，再切到 diff 对比表〕

第二天，分析师让 agent 重写一遍承付义务段——这次她什么都没说。

v4 第一次写出来的是什麼？三段结构——先按构成拆分（区分现金 vs 股本）、再按期限分布（0-12 / 12-24 / 24+）、最后给 so-what。其中关键校正一句话点穿：外界会把 EchoStar 196 亿当成"近期要现金出去 196 亿"，但其中只有最高 85 亿是现金，且要到 2027-11-30 才付；111 亿是股本稀释——这两件事的财务含义完全不同。

更关键的是 so-what：未来 18 个月并不构成现金压力高峰；真正的压力点集中在 2027 Q3-Q4，因此 IPO 时点窗口大概率被 2027 Q3 的现金需求倒逼。

〔切到 diff 对比表〕

跟 v3 第一次产出比，**v4 第一次产出已经把上一轮分析师才"说出来"的所有判断都写进去了**。这一轮的修改指令数：**0 条**——她不需要再说了。

4.9 一句话结论 (15 秒)

〔页面：STAGE 8 big-quote〕

那句"承付义务太浅，拆出期限分布"，没有被她自己再说第二遍。它被 **instructions** 表接住、被反思器抽象成"大额数字必拆结构"、过留出集、变成 **skill v4** 的拷问清单第 7 条——对所有未来同类调研永久生效。

5 · 收束 (1 分钟)

〔页面：滚到 CTA 区〕

回到一开始那个问题——为什么要做这件事？三句话：

第一，你嘴里那句"该这样改"，今天一次性使用、明天就消失了。一年下来你说了几百遍同一个意思。浪费的不是 **token**，是你这个本应做更高阶判断的人的时间。

第二，你说的话只要被记住，就会以复利方式回报你。一句话被存一次，循环跑一万次，这一句话就替你做了一万次决定。这是非线性的杠杆。

第三，五层循环 + 进化引擎合起来，就是把你这个"调研员"蒸馏成一个永远在岗的智能体。Bttr 那句话很合用——*Senior people will increasingly act like fund partners. Outcome over output.* 你不需要再亲手把每一份调研改完，你只需要让你说过的每一句话都被记住、被沉淀、被复用。

而你，坐在循环的边缘，做循环做不了的事——新情境下的判断、和客户的谈判、对一份报告的品味判断、深夜里关于公司方向的犹豫。

The company is a brain in the middle. Humans sit at the edge.

谢谢，开始问答。